

APUNTES Microcontroladores Y Microprocesadores

1. DIFERENCIA ENTRE MICROPROCESADOR Y MICROCONTROLADOR

El **microprocesador solo** incluye **CPU**. Es decir, **necesita memoria** RAM, ROM **y periféricos** externos. Por ello puede ser más **potente** y **flexible**. Se usa en ordenadores. [RASPBERRY PI]

El **microcontrolador** incluye **todo (CPU + memoria + periféricos** integrados) en un mismo **chip**. Es de **bajo consumo** y **bajo coste**. Se usa en Arduino, sistemas embebidos y automatización. [ARDUINO]

1.5 DIFERENCIAS ENTRE ARDUINO VS RASPBERRY

Arduino	Raspberry
Microcontrolador	Microprocesador
Tiempo real	NO tiempo real
Sin SO	Linux
Muy simple	Más potente
C/C++	Python/Linux
Control hardware	Aplicaciones complejas

2. QUE ES IOT

IoT (Internet of Things) es un concepto que describe la **conexión de dispositivos físicos** a Internet para que puedan recopilar, enviar y recibir datos automáticamente. Estos dispositivos suelen incorporar sensores, microcontroladores, módulos de comunicación, etc... con el objetivo es permitir monitorización, automatización y control remoto de sistemas físicos.

3. ARQUITECTURA AVR VS ARM

AVR [Arquitectura de **8 bits**] es una familia de microcontroladores de tipo **RISC** (Reduced Instruction Set Computer) de **alto rendimiento** y **bajo consumo**. Utilizada en Arduino Uno (ATmega328P).

⚠ La **estructura RISC** se caracteriza por instrucciones simples y registros de 8 bits.

ARM [Arquitectura de **32 bits**] es la arquitectura de 32 y 64 bits, tiene **mayor potencia**, **velocidad**, más **memoria** y **capacidad de proceso**. Usada en Raspberry Pi y Arduino Due.

4. BLOQUES DE LA ARQUITECTURA ARM

- **CPU**: Ejecuta instrucciones, opera y controla el resto de bloques.

- **Memoria:** Almacena programas, datos y variables
 - **RAM** → datos **temporales** y volátil.
 - **ROM** → datos de **lectura permanentes**
 - **Flash** → tipo **EEPROM**, guarda el programa del microcontrolador y es reprogramable
- **GPIO:** Pines de entrada (digitalRead()) y salida(digitalWrite()).
 - ⚠ Una señal **analógica** es continua y varía infinitamente en el tiempo, ideal para magnitudes físicas. Una señal **digital** es discreta y salta entre dos estados (0 y 1)
- **Temporizadores:** Circuitos integrados que cuentan **pulsos de reloj**, controlan tiempos, **PWM (Modulación por Ancho de Pulsos)** e **interrupciones (ISR)**.

⚠ **Interrupciones / ISR** nos permiten reaccionar inmediatamente a eventos (sensor PIR, botón...)

Ejemplo: attachInterrupt(digitalPinToInterrupt(2), funcion, RISING);

- **attachInterrupt(...):** Activa la función de interrupción en el sistema
- **digitalPinToInterrupt(2):** Traduce el número de pin físico (2) al número de interrupción
- **funcion:** Es el nombre de la función que se ejecutará cuando ocurra el evento
- **RISING:** Significa que la interrupción se disparará cuando el pin cambie de LOW a HIGH

⚠ **PWM** es una **técnica para simular señales analógicas usando señales digitales** rápidas.

Ejemplo: analogWrite (9,128)

- **Interfaces de comunicación**

Protocolo	Hilos (Cables)	Velocidad	Dispositivos	Complejidad
UART	2	Media	Punto a punto	Baja
I2C	2	Media	Muchos	Media
SPI	4	Muy alta	Varios	Alta

- **UART:** Comunicación **serie (bit a bit)** simple, usa **pines TX** transmite y **RX** recibe
- **SPI:** Comunicación **serie rápida y con más cable: MOSI** (maestro-esclavo), **MISO** (esclavo-maestro), **SCK** y **SS** (selección de dispositivo). **Full dúplex. Maestro (controla) /esclavo(espera órdenes)**
- **I2C:** Comunicación **serie síncrona (compartiendo reloj)**. Usa solo **2 cables: SDA** (data), **SCL** (clock). **Maestro/esclavo.**
- **USB**

⚠ **Half Dúplex (walkie-talkie)** indica que la **comunicación** va en **ambos sentidos** y en **full dúplex** envían y reciben al **mismo tiempo (teléfono)**

- **ADC/DAC:** Conversión analógica-digital y digital-analógica. *Ejemplo: analogRead(A0);*

5. SISTEMAS DE ALIMENTACIÓN EN ARDUINO

USB: 5V. Alimentación desde ordenador.

Jack externo: 7V–12V recomendados. Con un **regulador interno** baja a 5V.

Pin VIN: Entrada de alimentación externa.

Pines 5V y 3.3V: Salida **regulada** para sensores y módulos.

6. MEMORIAS EN MICROCONTROLADORES AVR

Flash: Guarda el programa. Es **no volátil**.

SRAM: Variables temporales. **Volátil**.

EEPROM: Datos permanentes. **No volátil**.

7. CICLO DE INSTRUCCIÓN

1. Fetch: Se **busca** la **instrucción en memoria** y la carga internamente [\[ADD D0,D1\]](#)

2. Decode: La **CPU interpreta** la instrucción.

3. Execute: Se **ejecuta** la operación.

4. Write Back: Se **guarda el resultado**.

8. FÓRMULAS IMPORTANTES

LM35: Sensor Analógico de temperatura. Convierte la temperatura en voltaje

$$T(^{\circ}C) = \frac{(analogRead(...) \cdot 5.0 \cdot 100)}{1023}$$

Ejemplo:

```
#define LM35 A1

void setup() {
  Serial.begin(9600); #baud rate: cantidad de bits en 1 seg
}

void loop() {
  int lectura = analogRead(LM35);
  float temperatura = lectura * 5.0 * 100 / 1023;

  Serial.print("Temperatura: ");
  Serial.println(temperatura);

  delay(1000);
}
```

9. LDR: Resistencia que cambia según la luz. Se usa para detectar si hay mucha o poca luz

Ejemplo:

```
#define LDR A0
#define LED 9

void setup() {
    pinMode(LED, OUTPUT);
    Serial.begin(9600);
}

void loop() {
    int luz = analogRead(LDR);

    if (luz < 300) {
        digitalWrite(LED, HIGH);
    } else {
        digitalWrite(LED, LOW);
    }

    Serial.println(luz);
    delay(1000);
}
```

10. SENSOR PIR: Sensor digital de movimiento. HIGH → movimiento, LOW → no movimiento

Ejemplo:

```
#define PIR 2
#define LED 13

void setup() {
    pinMode(PIR, INPUT);
    pinMode(LED, OUTPUT);
    Serial.begin(9600);
}

void loop() {
    int movimiento = digitalRead(PIR);

    if (movimiento == HIGH) {
        digitalWrite(LED, HIGH);
        Serial.println("Movimiento detectado");
    } else {
        digitalWrite(LED, LOW);
        Serial.println("Sin movimiento");
    }

    delay(500);
}
```

```
}
```

11. POTENCIÓMETRO: Resistencia variable. Se usa como ruleta o regulador y se conecta a entrada analógica. *Devuelve valores 0–1023.*

Ejemplo:

```
#define POT A0

void setup() {
    Serial.begin(9600);
}

void loop() {
    int valor = analogRead(POT);

    Serial.println(valor);

    delay(500);
}
```

12. BUZZER: Zumbador. HIGH → suena, LOW → se apaga

Ejemplo:

```
#define BUZZER 8

void setup() {
    pinMode(BUZZER, OUTPUT);
}

void loop() {
    digitalWrite(BUZZER, HIGH);
    delay(1000);

    digitalWrite(BUZZER, LOW);
    delay(1000);
}
```

13. RELÉ: Interruptor controlado eléctricamente. HIGH → activa relé, LOW → desactiva

Ejemplo:

```
#define RELE 7

void setup() {
    pinMode(RELE, OUTPUT);
}

void loop() {
```

```
digitalWrite(RELE, HIGH);  
delay(2000);  
  
digitalWrite(RELE, LOW);  
delay(2000);  
}
```

14. LCD I2C: Pantalla LCD que utiliza protocolo I2C para reducir cables

Ejemplo:

```
#include <Wire.h>  
#include <LiquidCrystal_I2C.h>  
  
LiquidCrystal_I2C lcd(0x27,16,2);  
  
void setup() {  
    lcd.init();  
    lcd.backlight();  
    lcd.clear();  
    lcd.setCursor(0,1);  
    lcd.print("Hola");  
}  
  
void loop() {  
}
```

15. HC-SR04: Sensor ultrasónico que mide **distancias** usando ultrasonidos

(TRIG → envía pulso y ECHO → recibe rebote)

Ejemplo:

```
#define TRIG 9  
#define ECHO 10  
  
void setup() {  
    pinMode(TRIG, OUTPUT);  
    pinMode(ECHO, INPUT);  
    Serial.begin(9600);  
}  
  
void loop() {  
  
    digitalWrite(TRIG, LOW);  
    delayMicroseconds(2);  
  
    digitalWrite(TRIG, HIGH);  
    delayMicroseconds(10);  
  
    digitalWrite(TRIG, LOW);
```

```

    long duracion = pulseIn(ECHO, HIGH);

    float distancia = duracion * 0.034 / 2;

    Serial.println(distancia);

    delay(500);
}

```

16. DHT11/DHT22: Sensores digitales de temperatura y humedad

DHT11	DHT22
Menos preciso	Más preciso
Más barato	Más caro
Menor rango	Mayor rango

Ejemplo:

```

#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
    Serial.begin(9600);
    dht.begin();
}

void loop() {

    float t = dht.readTemperature();
    float h = dht.readHumidity();

    Serial.print("Temp: ");
    Serial.println(t);

    Serial.print("Humedad: ");
    Serial.println(h);

    delay(2000);
}

```

17. MQ2: Sensor de gas y humo. Detecta gases inflamables y humo.

Ejemplo:

```
#define MQ2 A0

void setup() {
    Serial.begin(9600);
}

void loop() {

    int gas = analogRead(MQ2);

    Serial.println(gas);

    delay(1000);
}
```

18. SERVOMOTOR: Motor que permite controlar **posición angular** mediante PWM.

Ejemplo:

```
#include <Servo.h>

Servo servo;

void setup() {
    servo.attach(9);
}

void loop() {

    servo.write(0);
    delay(1000);

    servo.write(90);
    delay(1000);

    servo.write(180);
    delay(1000);
}
```

19. ESTRUCTURA BÁSICA ARDUINO

```
void setup()
{
}

void loop()
{
}
```

20. FUNCIONES ARDUINO

1. Librerías

```
#include <LiquidCrystal_I2C.h>
#include <Wire.h>
```

2. Definir pines

```
#define LED 7
#define SENSOR A0
```

3. Setup

```
pinMode(LED, OUTPUT); /* pinMode(pin, INPUT / OUTPUT
Serial.begin(9600); /* inicia comunicacion serial
```

4. Loop

Entradas

```
digitalRead(pin); /* lee pin
int valor = analogRead(pin); /* lee valor analógico
[0-1023]
```

Salidas

```
digitalWrite(pin, HIGH); /* escribe HIGH/LOW en un pin
analogWrite(pin, valorPWM); /* escribe señal PWM [0-255]

Serial.print("Valor: "); /* imprime sin \n
Serial.println(valor); /* imprime con \n
```

Tiempo

```
delay(1000);
millis(); /* devuelve el tiempo desde startup

//unsigned long tiempo = millis();
```

If / else

```
if (valor > 500) {
    digitalWrite(LED, HIGH);
} else {
    digitalWrite(LED, LOW);
}
```

For /while

```
for (int i = 0; i < 10; i++) {
    Serial.println(i);
}
```

 **FUNCIÓN MAP:** Convierte un rango en otro.

Ejemplo: `int pwm = map(valor, 0, 1023, 0, 255);`

21. EJEMPLO ARDUINO

```
# Programa de ejemplo: sensor digital, LED y buzzer

#define PIR 2
#define LED 13
#define BUZZER 8

unsigned long tiempoAnterior = 0;
const long intervalo = 3000;
bool alarmaActiva = false;

void setup() {
    pinMode(PIR, INPUT);
    pinMode(LED, OUTPUT);
    pinMode(BUZZER, OUTPUT);
    Serial.begin(9600);
}

void loop() {
    int movimiento = digitalRead(PIR);
    if (movimiento == HIGH) {
        digitalWrite(LED, HIGH);
        digitalWrite(BUZZER, HIGH);
        tiempoAnterior = millis();
        alarmaActiva = true;
        Serial.println("Movimiento detectado");
    }
    if (alarmaActiva && millis() - tiempoAnterior >= intervalo)
    {

        digitalWrite(LED, LOW);
        digitalWrite(BUZZER, LOW);
        alarmaActiva = false;
        Serial.println("Alarma apagada");
    }
}
```

22. RASPBERRY PI GPIO

1. Librería

```
import RPi.GPIO as GPIO
```

2. Numeración BCM

```
GPIO.setmode(GPIO.BCM) #usa la numeración GPIO
```

3. Configuración de pines

Entrada

```
GPIO.setup(17, GPIO.IN) #configura pin de entrada
```

Salida

```
GPIO.setup(27, GPIO.OUT)
```

4.Lectura y escritura GPIO

Leer

```
GPIO.input(17) #lee pin
```

Escribir

```
GPIO.output(27, GPIO.HIGH) #escribe pin en HIGH o LOW  
GPIO.output(27, GPIO.LOW)
```

5. Programa

Delays

```
import time  
time.sleep(1) #pausa
```

Loop infinito

```
while True:
```

If/else

```
if estado == GPIO.HIGH:  
    print("Activo")  
else:  
    print("Inactivo")
```

for

```
for i in range(10):  
    print(i)
```

try/except

```
try:  
    while True:  
        pass  
except KeyboardInterrupt:  
    GPIO.cleanup() # LIMPIEZA GPIO ¡!
```

23. EJEMPLO PYTHON RASPBERRY

```
# Programa de ejemplo: sensor digital, LED y buzzer
```

```
import RPi.GPIO as GPIO
import time

SENSOR = 17
LED = 22
BUZZER = 27

GPIO.setmode(GPIO.BCM)

GPIO.setup(SENSOR, GPIO.IN)
GPIO.setup(LED, GPIO.OUT)
GPIO.setup(BUZZER, GPIO.OUT)

try:
    while True:
        estado = GPIO.input(SENSOR)

        if estado == GPIO.HIGH:
            GPIO.output(LED, GPIO.HIGH)

            GPIO.output(BUZZER, GPIO.HIGH)
            time.sleep(0.2)

            GPIO.output(BUZZER, GPIO.LOW)
            time.sleep(0.3)

            print("Sensor activado")

        else:
            GPIO.output(LED, GPIO.LOW)
            GPIO.output(BUZZER, GPIO.LOW)

            print("Sensor desactivado")

        time.sleep(1)

except KeyboardInterrupt:
    GPIO.output(LED, GPIO.LOW)
    GPIO.output(BUZZER, GPIO.LOW)
    GPIO.cleanup()
```